DS HW 7

1. Construct an Expression Tree

```
Aim: Construct an expression tree for the expression (a-b)+(c*d).

#include <stdio.h>
#include <stdlib.h>

struct Node {
    char data;
    struct Node *left, *right;
};

struct Node* createNode(char data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

void inorder(struct Node* root) {
    if (root != NULL) {
        if (root->left != NULL || root->right != NULL)
            printf("(");
        inorder(root->left);
        printf("%c", root->data);
        inorder(root->right);
        if (root->left != NULL || root->right != NULL)
            printf(")");
    }
}

int main() {
    struct Node* root = createNode('+');
    root->left = createNode('-');
    root->left->left = createNode('a');
    root->left->right = createNode('b');
    root->right = createNode('*');
    root->right->left = createNode('c');
    root->right->right = createNode('d');

    printf("Inorder Expression: ");
    inorder(root);
    printf("\n");
    return 0;
}

Sample Output:
Inorder Expression: ((a-b)+(c*d))
```

2. Linear Search and Binary Search

```
Aim: Perform linear search and binary search on a sorted array.

#include <stdio.h>

int main() {
    int a[100], n, key, i;
    int linear = -1, low, high, mid, binary = -1;

    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements in sorted order: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Enter element to search: ");
    scanf("%d", &key);

    for (i = 0; i < n; i++) {
        if (a[i] == key) {
            linear = i;
            break;
        }
    }

    if (linear != -1)
        printf("Linear Search: Found at position %d\n", linear + 1);
    else
        printf("Linear Search: Not found\n");

    low = 0;
```

```
    high = n - 1;
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == key) {
            binary = mid;
            break;
        } else if (a[mid] < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    if (binary != -1)
        printf("Binary Search: Found at position %d\n", binary + 1);
    else
        printf("Binary Search: Not found\n");
    return 0;
}
```

```
Sample Output:
Enter number of elements: 5
Enter elements in sorted order: 10 20 30 40 50
Enter element to search: 30
Linear Search: Found at position 3
Binary Search: Found at position 3
```

Note: Binary search requires the array to be sorted.

3. Stack Using a Linked List

```
Aim: Implement push, pop, and display operations using a linked-list stack.

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* top = NULL;

void push(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Stack Overflow\n");
        return;
    }
    newNode->data = value;
    newNode->next = top;
    top = newNode;
    printf("%d pushed\n", value);
}

void pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }
    struct Node* temp = top;
    printf("Popped element: %d\n", top->data);
    top = top->next;
    free(temp);
}

void display() {
    struct Node* temp = top;
    if (temp == NULL) {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    push(10);
    push(20);
    push(30);
    display();
    pop();
    display();
    return 0;
}

Sample Output:
10 pushed
20 pushed
30 pushed
Stack: 30 20 10
Popped element: 30
Stack: 20 10
```

4. Cycle Detection in an Undirected Graph

```
Aim: Check whether an undirected graph contains a cycle using DFS.

#include <stdio.h>

int graph[20][20], visited[20], n;

int hasCycle(int v, int parent) {
    visited[v] = 1;
    for (int i = 0; i < n; i++) {
        if (graph[v][i]) {
            if (!visited[i]) {
                if (hasCycle(i, v))
                    return 1;
            } else if (i != parent) {
```

```
                return 1;
            }
        }
    }
    return 0;
}

int main() {
    int edges, u, v;
    printf("Enter number of vertices and edges: ");
    scanf("%d%d", &n, &edges);

    for (int i = 0; i < edges; i++) {
        scanf("%d%d", &u, &v);
        graph[u][v] = 1;
        graph[v][u] = 1;
    }

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    for (int i = 0; i < n; i++) {
        if (!visited[i] && hasCycle(i, -1)) {
            printf("Graph contains a cycle\n");
            return 0;
        }
    }
    printf("Graph does not contain a cycle\n");
    return 0;
}

Sample Output:
Enter number of vertices and edges: 3 3
0 1
1 2
2 0
Graph contains a cycle

Note: Vertices are numbered 0 to n-1. Assumes no self-loops or parallel edges.
```

5. Count the Number of Islands in a Binary Matrix

Aim: Count connected groups of 1s using 8-directional DFS.

```
#include <stdio.h>

int a[20][20], visited[20][20];
int rows, cols;
int dx[] = {-1, -1, -1, 0, 0, 1, 1, 1};
int dy[] = {-1, 0, 1, -1, 1, -1, 0, 1};

void dfs(int x, int y) {
    visited[x][y] = 1;
    for (int k = 0; k < 8; k++) {
        int nx = x + dx[k];
        int ny = y + dy[k];
        if (nx >= 0 && nx < rows && ny >= 0 && ny < cols &&
            a[nx][ny] == 1 && !visited[nx][ny]) {
            dfs(nx, ny);
        }
    }
}

int main() {
    int count = 0;
    printf("Enter rows and columns: ");
    scanf("%d%d", &rows, &cols);
    printf("Enter matrix (0 or 1):\n");

    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            scanf("%d", &a[i][j]);

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (a[i][j] == 1 && !visited[i][j]) {
                dfs(i, j);
                count++;
            }
        }
    }
    printf("Number of islands: %d\n", count);
    return 0;
}

Sample Output:
Enter rows and columns: 4 4
Enter matrix (0 or 1):
1 1 0 0
0 1 0 1
0 0 1 1
1 0 0 0
Number of islands: 3

Note: This uses 8-directional connectivity (diagonal cells also connect).
```

6. Minimum Spanning Tree Using Kruskal's Algorithm

Aim: Find the minimum spanning tree and its total cost using Kruskal's algorithm.

```
#include <stdio.h>

struct Edge {
    int u, v, w;
};

struct Edge e[100], temp;
int parent[100];

int find(int x) {
    while (parent[x] != x) {
        parent[x] = parent[parent[x]];
        x = parent[x];
    }
    return x;
}

int main() {
    int n, m, cost = 0, count = 0;
    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &m);
    printf("Enter edges (u v weight):\n");

    for (int i = 0; i < m; i++)
        scanf("%d%d%d", &e[i].u, &e[i].v, &e[i].w);
```

```
    for (int i = 0; i < m - 1; i++) {
        for (int j = 0; j < m - i - 1; j++) {
            if (e[j].w > e[j + 1].w) {
                temp = e[j];
                e[j] = e[j + 1];
                e[j + 1] = temp;
            }
        }
    }

    for (int i = 0; i < n; i++)
        parent[i] = i;

    printf("Edges in Minimum Spanning Tree:\n");
    for (int i = 0; i < m; i++) {
        int a = find(e[i].u);
        int b = find(e[i].v);
        if (a != b) {
            printf("%d - %d : %d\n", e[i].u, e[i].v, e[i].w);
            parent[a] = b;
            cost += e[i].w;
            count++;
        }
    }

    if (count != n - 1)
        printf("Graph is disconnected; MST does not exist\n");
    else
        printf("Minimum cost: %d\n", cost);
    return 0;
}
```

```
Sample Output:
Enter vertices and edges: 4 5
Enter edges (u v weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
Edges in Minimum Spanning Tree:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Minimum cost: 19
```

7. Dijkstra's Shortest Path Algorithm

Aim: Find the shortest distances from a source vertex to all other vertices.

```
#include <stdio.h>

#define INF 999999

int main() {
    int n, source;
    int graph[20][20], dist[20], visited[20] = {0};

    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix (0 means no edge):\n");

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
            if (i != j && graph[i][j] == 0)
                graph[i][j] = INF;
        }
    }

    printf("Enter source vertex: ");
    scanf("%d", &source);
    for (int i = 0; i < n; i++)
        dist[i] = INF;
    dist[source] = 0;

    for (int count = 0; count < n; count++) {
        int u = -1, min = INF;
        for (int i = 0; i < n; i++) {
            if (!visited[i] && dist[i] < min) {
                min = dist[i];
                u = i;
            }
        }
        if (u == -1)
            break;
        visited[u] = 1;

        for (int v = 0; v < n; v++) {
            if (!visited[v] && graph[u][v] != INF &&
                dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("Shortest distances from vertex %d:\n", source);
    for (int i = 0; i < n; i++) {
        if (dist[i] == INF)
            printf("To %d: Unreachable\n", i);
        else
            printf("To %d: %d\n", i, dist[i]);
    }
    return 0;
}
```

```
Sample Output:
Enter number of vertices: 4
Enter adjacency matrix (0 means no edge):
0 5 10 0
5 0 3 11
10 3 0 2
0 11 2 0
Enter source vertex: 0
Shortest distances from vertex 0:
To 0: 0
To 1: 5
To 2: 8
To 3: 10
```

Note: Edge weights must be non-negative.

8. Find a Mother Vertex in a Directed Graph

Aim: Find a vertex from which all vertices are reachable.

```
#include <stdio.h>

int graph[20][20], visited[20], n;

void dfs(int v) {
```

```
    visited[v] = 1;
    for (int i = 0; i < n; i++)
        if (graph[v][i] && !visited[i])
            dfs(i);
}

int main() {
    int m, u, v, candidate = -1;
    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &m);
    printf("Enter directed edges (u v):\n");

    for (int i = 0; i < m; i++) {
        scanf("%d%d", &u, &v);
        graph[u][v] = 1;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            dfs(i);
            candidate = i;
        }
    }

    for (int i = 0; i < n; i++)
        visited[i] = 0;
    dfs(candidate);

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            printf("No mother vertex exists\n");
            return 0;
        }
    }
    printf("Mother vertex: %d\n", candidate);
    return 0;
}

Sample Output:
Enter vertices and edges: 4 4
Enter directed edges (u v):
0 1
1 2
2 0
2 3
Mother vertex: 0
```

9. Find Strongly Connected Components (Kosaraju's Algorithm)

```
Aim: Find all strongly connected components in a directed graph.

#include <stdio.h>

int graph[20][20], transpose[20][20];
int visited[20], stack[20], top = -1, n;

void dfs1(int v) {
    visited[v] = 1;
    for (int i = 0; i < n; i++)
        if (graph[v][i] && !visited[i])
            dfs1(i);
    stack[++top] = v;
}

void dfs2(int v) {
    visited[v] = 1;
    printf("%d ", v);
    for (int i = 0; i < n; i++)
        if (transpose[v][i] && !visited[i])
            dfs2(i);
}

int main() {
    int m, u, v;
    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &m);
    printf("Enter directed edges (u v):\n");

    for (int i = 0; i < m; i++) {
        scanf("%d%d", &u, &v);
        graph[u][v] = 1;
        transpose[v][u] = 1;
    }

    for (int i = 0; i < n; i++)
        if (!visited[i])
            dfs1(i);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    printf("Strongly Connected Components:\n");
    while (top >= 0) {
        int v = stack[top--];
        if (!visited[v]) {
            dfs2(v);
            printf("\n");
        }
    }
    return 0;
}

Sample Output:
Enter vertices and edges: 5 6
Enter directed edges (u v):
0 1
1 0
1 2
2 3
3 4
4 2
Strongly Connected Components:
0 1
2 4 3
```

10. Check Whether a Graph Is Bipartite

```
Aim: Check whether an undirected graph can be colored using two colors so adjacent vertices have
different colors.

#include <stdio.h>

int graph[20][20], color[20], n;

int bfs(int start) {
    int queue[20], front = 0, rear = 0;
    queue[rear++] = start;
    color[start] = 0;

    while (front < rear) {
        int u = queue[front++];
        for (int v = 0; v < n; v++) {
```

```
            if (graph[u][v]) {
                if (color[v] == -1) {
                    color[v] = 1 - color[u];
                    queue[rear++] = v;
                } else if (color[v] == color[u]) {
                    return 0;
                }
            }
        }
    }
    return 1;
}

int main() {
    int m, u, v;
    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &m);
    printf("Enter undirected edges (u v):\n");

    for (int i = 0; i < m; i++) {
        scanf("%d%d", &u, &v);
        graph[u][v] = 1;
        graph[v][u] = 1;
    }

    for (int i = 0; i < n; i++)
        color[i] = -1;

    for (int i = 0; i < n; i++) {
        if (color[i] == -1 && !bfs(i)) {
            printf("Graph is not bipartite\n");
            return 0;
        }
    }
    printf("Graph is bipartite\n");
    return 0;
}

Sample Output:
Enter vertices and edges: 4 4
Enter undirected edges (u v):
0 1
1 2
2 3
3 0
Graph is bipartite
```